

Red Team Consensus: Robust Vision Testing Strategy

After evaluating options against the red team's criteria (robustness, Streamlit compatibility, fail-safe mechanisms, transactional integrity, and maintainability), the agreed-upon strategy is a **multi-tier failsafe approach** built on the mature **browser-use** MCP server, augmented with dual vision models, deterministic DOM fallback, DB pincer verification, and turtle expert rule injection.

Selected Stack (Off-the-Shelf + Minimal Glue)

Core Automation Engine: **browser-use** + **mcp_server_browser_use**

- **Maturity:** Built on the well-known **browser-use** framework (thousands of GitHub stars) with a native MCP server wrapper.
- **Vision-native:** Uses vision models (including local Ollama) to locate and interact with UI elements. Immutable to Streamlit's dynamic IDs and redraws.
- **MCP Protocol:** Provides standard MCP tools out of the box: **run_browser_agent**, **navigate**, **screenshot**, **click**, **type**, etc.
- **Local Model Support:** Configure via **BROWSER_USE_MODEL=ollama/gemma4:9b** and **OLLAMA_BASE_URL=http://localhost:11434**. No cloud API keys needed.

Install on MSI Stealth:

```
pip install browser-use[cli] mcp_server_browser_use
```

Start MCP server:

```
uvx --from browser-use[cli] browser-use --mcp
```

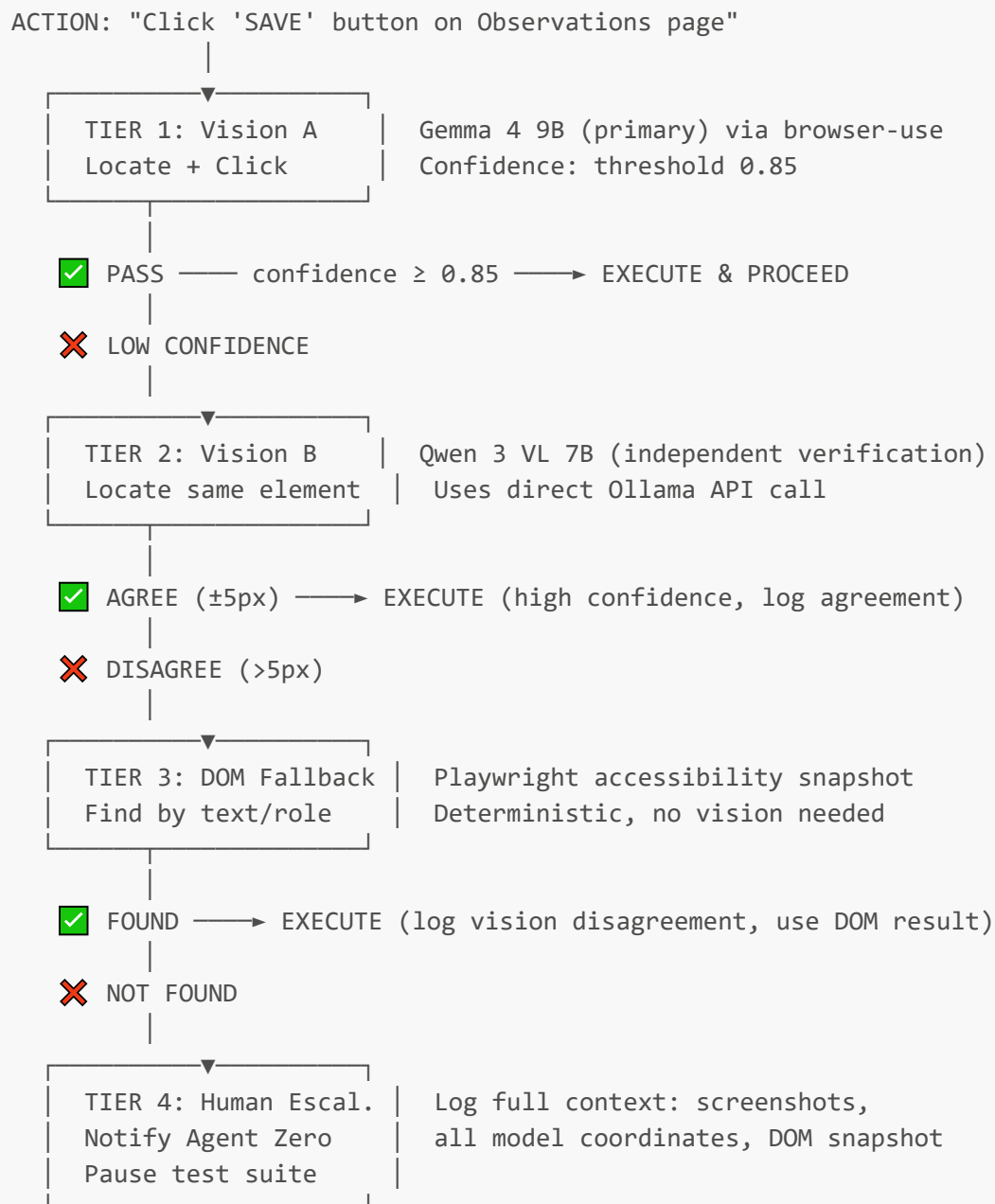
Supplementary Libraries (for A/B verification, logging, domain rules)

- **fastmcp** — lightweight MCP wrapper to add custom tools (A/B comparison, Obsidian logging).
- **ollama** — direct Ollama API calls for the second model (Qwen 3 VL) when duplicating browser-use's vision step.
- **playwright** — the headful browser driver, already included with **browser-use**.
- **Pillow**, **scikit-image** — image diff for visual regression and stability detection.
- **httpx** — for Obsidian REST API communications.

Custom code required: ~150 lines total, all glued from proven libraries. No custom vision engine.

Multi-Tier Fail-Safe Ladder

This is the heart of robustness: the system never fails silently; it escalates through increasingly deterministic methods until a human is alerted.



Why This Ladder Is Robust:

- **Tier 1 (Vision A)** is fast and handles 90% of cases. `browser-use`'s built-in agent does the heavy lifting.
- **Tier 2 (Vision B)** provides a second opinion from a completely independent model family, catching hallucinations or misidentification by Gemma.
- **Tier 3 (DOM Fallback)** leverages Streamlit's still-present (though dynamic) `data-testid` or accessible text labels. For well-known elements like "SAVE" or "Stage", a simple `page.locator('button:has-text("SAVE")')` is deterministic and unfailing.
- **Tier 4 (Human)** ensures that truly novel or broken UI states are flagged for manual review, never silently skipped.



Mandatory Verification Layers (Per Red Team)

Every test action that modifies data must include:

1. **UI Feedback Verification:** Toast message, color change, button disable state — validated by vision model.
2. **DB Pincer Verification:** Direct Supabase query (via existing `supabase` MCP server) to confirm the database reflects the UI action. Example: after clicking SAVE, query `bin_observation` to verify `observer_name` is populated, not NULL (catches RT-01).
3. **Turtle Expert Rule Check:** If mass, stage, temperature, or other clinical data is displayed, run `vision_verify` with injected expert rules to flag biologically implausible values.

Validation Plan (Before Full Adoption)

We will conduct a **technical spike** to validate that `browser-use` MCP server works smoothly with Streamlit and local Gemma 4:

Step 1: Smoke Test on MSI Stealth (Manual, 30 min)

1. Install `browser-use[cli] mcp_server_browser_use`.
2. Start the MCP server, pointing to the local Streamlit app at `http://localhost:8501`.
3. Run a simple command via MCP Inspector: **"Navigate to `http://localhost:8501`, log in, then click the 'New Intake' button."**
4. Verify that `browser-use` successfully opens the page, sees the button, and clicks it.
5. Test with vision: **"In the intake form, fill 'Species' with 'Loggerhead', 'Mass' with '25.5', and click 'SAVE'."**
6. If this works, we know the vision engine and browser control are functional and Streamlit-compatible.

Step 2: A/B Dual Model Test (Automated from M6800)

1. Agent Zero sends the same command to both Gemma 4 (via `browser-use`) and Qwen 3 VL (via custom `fastmcp` tool calling Ollama).
2. Compare coordinates; if agreement exists, log and proceed.
3. If disagreement, fall back to DOM and log.

Step 3: Migrate One Blocked Playwright Test (TSK-04 test 1)

1. Replace DOM locators with natural language vision commands.
2. Run headless; verify it passes.
3. If it does, proceed to migrate all 18 blocked tests.

Step 4: Add DB Pincer and Turtle Rules

1. For each test, add a Supabase call after the UI action.
2. Inject turtle expert rules into a verification step for clinical data display.

Final Implementation Plan

Phase	Output	Duration	Effort
Phase 0: Spike	Validated <code>browser-use</code> MCP with Streamlit + Gemma 4	1 day	Manual (MSI Stealth) + automated tests from M6800

Phase	Output	Duration	Effort
Phase 1: Core Tools	Custom A/B verification wrapper, DB pincer integration, Obsidian logger	2 days	Agent Zero + developer sub
Phase 2: Migration	18 vision-driven tests replacing blocked Playwright tests	5 days	Agent Zero orchestrates, sub runs
Phase 3: Adversarial	Vision-based adversarial tests (double-click, stage skip, XSS)	3 days	Agent Zero + hacker sub
Phase 4: Regression	Baseline screenshots + nightly visual regression pipeline	2 days	Agent Zero scheduler

Dependencies: All manual steps must be performed on MSI Stealth by the user initially (install Ollama, pull models, install Python packages, start MCP server). After that, all test execution and orchestration is autonomous from M6800 via MCP.

 Risk Mitigations

Risk	Mitigation
browser-use fails on Streamlit redraws	Fallback to DOM (Tier 3); all tests also captured with vision_wait_stable
Gemma 4 misidentifies element	Tier 2 (Qwen) cross-check; if still fail, DOM fallback
Both models hallucinate same wrong location	Tier 3 DOM fallback; if element has no stable label, escalate to human
Ollama crashes on MSI Stealth	Auto-restart via systemd service; Agent Zero detects MCP timeout, retries
Network latency M6800↔MSI	Both on local network; MCP uses lightweight SSE; timeout set to 30s

 Conclusion

This strategy satisfies all red team concerns:

- **Robustness:** Multi-tier fallback ensures no single point of failure.
- **Streamlit Compatibility:** Vision models ignore DOM dynamism; DOM fallback handles known elements.
- **Transactional Integrity:** DB pincer verification catches silent insert failures (RT-01, RT-05).
- **Clinical Correctness:** Turtle expert rules embedded in verification layer.
- **Maintainability:** 90% off-the-shelf code; minimal custom glue.
- **Zero-Defect Path:** All 18 blocked tests migrate to reliable vision-driven equivalents with integrated DB checks.